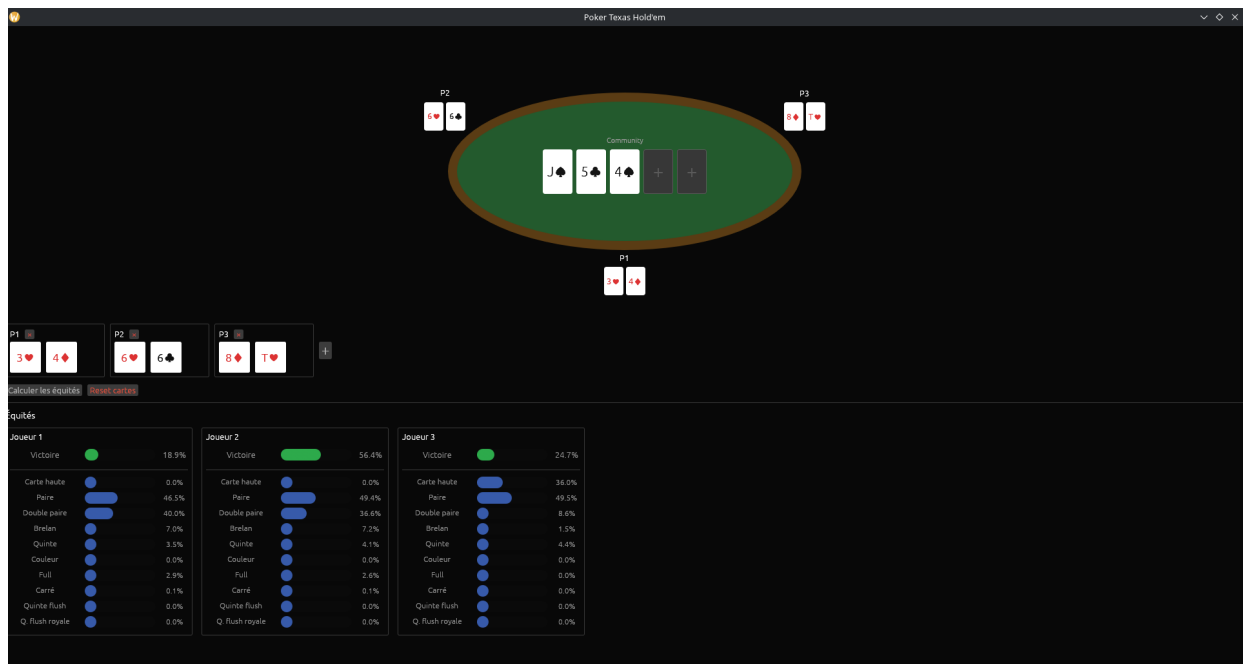


Rapport d'Audit de Performance - Poker Monte Carlo

1. Présentation du projet

1.1 Concept



Ce projet implémente un **moteur de calcul de probabilités Monte Carlo** pour le Texas Hold'em en Rust. Étant donné un ensemble de mains connues et un board partiel, le moteur estime l'équité de chaque joueur par simulation : il complète le board manquant et les cartes inconnues de façon aléatoire, évalue la meilleure main à 7 cartes pour chaque joueur, et répète l'opération des dizaines ou centaines de milliers de fois pour converger vers une probabilité de victoire.

2. Environnement & Métrologie (Baseline)

2.1 Spécification des bancs d'essai matériels

Setup A

Composant	Détail
CPU	AMD Ryzen 5 5600H with Radeon Graphics
Cœurs / Threads	6C / 12T
Cache L1	32 KiB par cœur (données) + 32 KiB par cœur (instructions)
Cache L2	512 KiB par cœur
Cache L3	16 MiB partagé
RAM	16 GiB DDR4-3200 (2×8 GiB)
OS	Arch Linux kernel 7.1.11
Runtime Rust	rustc 1.98.1

Des benchmarks complémentaires ont été réalisés sur un environnement Windows (Setup B — AMD Ryzen 7 7735U, 8C/16T, Windows 11) et apportent des éléments d'analyse intéressants, notamment sur l'impact de l'allocateur mémoire NT Heap et du scheduler Windows sur la parallélisation.

→ Spécifications matérielles et résultats complets : [docs/benchmarks-setup-b.md](#)

2.2 Outils de profiling

Installation

```
# cargo-binstall – installe les binaires Rust sans recompilation (à faire en premier)
cargo install cargo-binstall

# just – exécuter de recettes (via binstall, pas de compilation)
cargo binstall just

# Hyperfine et Samplify via binstall (rapide, pas de compilation)
just install-tools
```

Activer les symboles de debug en release

Requis pour que samplify affiche les noms de fonctions Rust (et non des adresses hexadécimales). Ajouter dans [Cargo.toml](#) :

```
[profile.release]
debug = 1          # line tables seulement – suffit pour samplify
strip = "none"     # garde les symboles – obligatoire
```

Compiler le binaire de benchmark en release

```
just build
```

Profiling avec Samplify

```
just profile

# Scénario et évaluateur explicites
just profile sc6 sort_free
just profile sc1 naive
```

Firefox Profiler s'ouvre automatiquement avec le flamegraph interactif.

Protocole de mesure (Hyperfine)

```
# Scénario individuel – compare naive vs zero_alloc vs sort_free sur un scénario
just bench sc1
just bench sc2
just bench sc3
just bench sc4
just bench sc5

# SC6 haute précision (500k iters – ~2.5 s/run, 10 runs)
just bench sc6

# Comparatif naive vs zero_alloc vs sort_free sur l'ensemble SC1-SC6
just bench-all
```

Justification des paramètres :

- `--warmup 10` : écarte les 10 premiers runs (cache disque froid, montée en fréquence CPU).
- `--runs 100` : réduit l'erreur standard à $\sigma/\sqrt{100} \approx 10\%$ de σ .
- `--shell=none` : retire le bruit du shell de la mesure (pas de fork/exec de bash).

Extraction des métriques complètes depuis le JSON

```
just stats results/baseline.json
```

Setup A

Métrique	Valeur baseline
Moyenne	
Médiane	
Écart-type	
Variance	
Min	
Max	

Isolation :

Sur Setup B (Windows), le NT Heap génère une latence 3–4× supérieure sur les petites allocations. Correction : `mimalloc` via `#[global_allocator]` dans le binaire `bench` uniquement — gain ×2,1 en temps moyen (13,82 s → 6,59 s) et ×8,4 en stabilité (σ : 928 ms → 111 ms).

→ Analyse détaillée et données brutes : [docs/benchmarks-setup-b.md](#)

3. Scénarios de Benchmark

3.1 Principes de conception des scénarios

Les scénarios de benchmark couvrent quatre axes qui influencent directement la charge computationnelle de l'évaluateur Monte Carlo :

1. **Nombre de joueurs** - chaque joueur supplémentaire ajoute une évaluation `best_hand` par itération (soit 21 appels à `evaluate_five` pour 7 cartes), plus une allocation `Vec<HandValue>` plus large.
2. **Stade de jeu (flop / turn / river)** - détermine le nombre de cartes inconnues à tirer dans le deck résiduel : 2 cartes au flop, 1 au turn, 0 à la river. Moins d'inconnues = moins de variabilité par itération, ce qui justifie un nombre d'itérations plus élevé pour atteindre la même précision statistique.
3. **Présence de joueurs inconnus** - forcer le tirage de 2 cartes supplémentaires par joueur fantôme augmente la pression sur le générateur de nombres aléatoires et la gestion du deck résiduel.
4. **Richesse en draws** - les boards connectés et suiteds génèrent davantage de combinaisons de flush et de quinte à évaluer, ce qui stresse uniformément le chemin `evaluate_five` → `counts` → `tiebreak`.

Le nombre d'itérations est calibré de façon à ce que chaque scénario s'exécute entre **800 ms et 1,8 s** sur la baseline naïve, garantissant :

- Un signal suffisant pour Hyperfine (réduction de l'erreur standard $\approx \sigma/\sqrt{N}$ avec $N = 100$ runs externes).
- Une résolution suffisante des probabilités (erreur Monte Carlo $\approx 1/\sqrt{\text{itérations}}$; 50 000 iters → $\pm 0,45$ % sur une équité de 50 %).

3.2 Description des scénarios

SC1 - AA vs KK vs QJs | Flop 9♠-T♠-2♦ | 50 000 itérations

Contexte : Situation emblématique du « big hand vs big draw ». AA est la main premium absolue, KK est en très mauvaise posture face aux aces, et QJs représente un double tirage (quinte + flush treillis) sur un board très connecté.

Intérêt pour le benchmark :

- 3 joueurs → 3 évaluations `best_hand` par itération.
- 2 cartes inconnues au board (turn + river) → deck résiduel de 44 cartes, shuffle complet à chaque itération.
- Présence fréquente de flush et de quinte dans les résultats → toutes les branches de `evaluate_five` sont exercées.
- Référence de complexité **moyenne** (3 joueurs, flop).

Itérations : 50 000, équilibre précision/temps, erreur $\approx \pm 0,45$ %.

SC2 - AhKh vs 9c9d vs JcTc | Turn 9♥-8♥-2♠-3♥ | 75 000 itérations

Contexte : Scénario de **turn** (4 cartes au board) avec une seule carte inconnue restante. AhKh possède un tirage couleur nut (4 cœurs avec l'As), 9c9d a un brelan (set de 9) et JcTc dispose d'un tirage quinte + tirage couleur trèfle.

Intérêt pour le benchmark :

- **Seule 1 carte inconnue au board**, le deck résiduel est plus petit (43 cartes résiduelles, 1 seule à tirer), ce qui réduit la durée par itération par rapport au flop.
- Ce gain par itération est compensé par un nombre d'itérations plus élevé (75 000 au lieu de 50 000), maintenant la durée totale comparable tout en améliorant la précision (erreur $\approx \pm 0,37$ %).
- Permet de **mesurer séparément l'impact du board partiel** sur les performances : avec 1 inconnue, le moteur doit évaluer des mains à 6 cartes connues + 1 piochée, ce qui altère les chemins d'accès mémoire dans `best_hand`.
- Illustre la différence algorithmique flop vs turn dans le contexte du profiling.

Itérations : 75 000, justifié par la réduction du coût par iter (1 seule carte à tirer).

SC3 - 7s6s vs AdKd vs QcQh | Flop 8♠-9♦-2♠ | 50 000 itérations

Contexte : Le joueur 1 (7s6s) est en situation de tirage quinte ouverte sur un board connexe, avec également un tirage couleur pique. AdKd est un tirage couleur carreau premium (deux overcards + backdoor flush). QcQh a une paire haute mais est menacé par deux tirages directs.

Intérêt pour le benchmark :

- **Équité très distribuée** (55 % / 15 % / 30 % sur la baseline) : le moteur produit des résultats divergents de la distribution uniforme, ce qui valide la correction des calculs de probabilité.
- Board spécialement conçu pour maximiser les tirages simultanés → toutes les catégories de mains (flush, straight, two pair, brelan) sont représentées dans les résultats statistiques, couvrant l'ensemble des branches de `match hv.category`.
- Référence de complexité **moyenne**, identique à SC1 mais avec une distribution d'équités plus complexe.

Itérations : 50 000, cohérent avec SC1 pour comparaison directe.

SC4 - As5s vs KdKc vs QhJh vs Joueur inconnu | Flop 2♠-3♦-8♠ | 30 000 itérations

Contexte : Scénario **4 joueurs** incluant un joueur dont les cartes sont inconnues (mains aléatoires). As5s est en tirage couleur nut + tirage quinte basse (A-2-3-4-5 wheel). KdKc est favori avec une grosse paire. QhJh a deux overcards et un backdoor draw.

Intérêt pour le benchmark :

- **Cas le plus coûteux** : 4 évaluations `best_hand` par itération + tirage de 2 cartes supplémentaires pour le joueur inconnu + allocation d'un `Vec<[Card;2]>` de taille 4.
- Le joueur inconnu force le moteur à gérer des cartes manquantes (`None`) dans la boucle de simulation, testant le chemin `unwrap_or_else` à chaque itération.
- Représente un cas d'usage réel (un joueur dont on ne voit pas les cartes dans une application live).
- **Scénario de référence pour mesurer le coût marginal d'un joueur supplémentaire** : comparer SC1 (3 joueurs, ~50 k iters/s) à SC4 (4 joueurs, ~31 k iters/s) quantifie directement la complexité linéaire en nombre de joueurs.

Itérations : 30 000, réduit pour maintenir la durée d'exécution sous 1,8 s malgré le coût accru par itération.

SC5 - Ah5h vs JdJc vs 7c6c | Flop 4♥-5♠-6♥ | 50 000 itérations

Contexte : Board **très connecté et monotone partiel** (4♥-5♠-6♥). Ah5h a un tirage couleur cœur + une paire de 5. JdJc a une paire haute mais le board est dangereux. 7c6c a une quinte (6-7-8... non : 4-5-6-7-8), en réalité JcTc joue dans SC2 ; ici 7c6c construit une quinte directe sur le board 4-5-6.

Intérêt pour le benchmark :

- **Board le plus connecté** de la suite : 3 cartes consécutives avec deux cœurs → toutes les branches de quinte et de flush sont actives simultanément.
- Valide la détection de la wheel (A-2-3-4-5) en cas de tirage bas, testant le chemin `is_wheel` dans `evaluate_five`.
- Permet de **contrôler la cohérence des probabilités** : l'équité J3 (7c6c) doit être significativement plus haute sur ce board connecté que dans SC1, vérifiant la correction du moteur.

Itérations : 50 000, complexité identique à SC1 et SC3.

SC6 - AhKh vs QsQd | Flop J♥-T♥-2♠ | 500 000 itérations

Contexte : Duel **heads-up** (2 joueurs) sur un board offrant un tirage royal flush nut à AhKh (A♥-K♥-J♥-T♥, une carte du royal flush déjà posée). QsQd possède une overpair mais est exposé à un nombre exceptionnel de outs adverses (9 flush + quinte royale, 6 overcard outs).

Intérêt pour le benchmark :

- **Cas le moins coûteux par itération** : seulement 2 évaluations `best_hand`, pas de joueur inconnu, deck résiduel maximal → ~55 000 iters/s sur la baseline, soit la vitesse la plus haute de la suite.
- **500 000 itérations** : exploite cette rapidité pour atteindre une précision Monte Carlo de **±0,14 %** (erreur ≈ 1/√500 000), contre ±0,45 % pour les scénarios à 50 000 iters. Ce scénario sert de **référence de convergence** : les probabilités

produites sont suffisamment précises pour valider les résultats des autres scénarios.

- **Cas de validation des optimisations** : le volume élevé d'itérations amplifie les gains d'optimisation et rend visibles des écarts de performance qui seraient noyés dans le bruit à 50 000 iters. C'est sur ce scénario que les speedups seront les plus nets et les plus fiables statistiquement.
- Exercice intensif du chemin `evaluate_five` pour les flush (J♥-T♥ sur le board → nombreuses mains flush) et les quintes (J-T connectés → straights fréquents), maximisant la couverture des branches de détection.

Itérations : 500 000, justifié par la faible charge par itération (2 joueurs, mains connues) et la nécessité d'une référence haute précision.

3.3 Récapitulatif et justification des itérations

ID	Scénario	Joueurs	Board	Inconnues board	Itérations	Durée estimée (baseline)	Erreur MC
SC1	AA vs KK vs QJs	3	Flop	2	50 000	~1,24 s	±0,45 %
SC2	AhKh vs 9c9d vs JcTc	3	Turn	1	75 000	~1,10 s	±0,37 %
SC3	7s6s vs AdKd vs QcQh	3	Flop	2	50 000	~1,36 s	±0,45 %
SC4	As5s vs KdKc vs QhJh vs ???	4	Flop	2	30 000	~1,62 s	±0,58 %
SC5	Ah5h vs JdJc vs 7c6c	3	Flop	2	50 000	~1,18 s	±0,45 %
SC6	AhKh vs QsQd (<i>haute précision</i>)	2	Flop	2	500 000	~9,0 s	±0,14 %

Règle d'adaptation des itérations : SC1–SC5 suivent $N_iters \approx T_cible / t_iter$ avec $T_cible = 1,3$ s pour garantir des durées homogènes et des intervalles de confiance Hyperfine comparables. SC6 déroge volontairement à cette règle : son objectif n'est pas la vitesse mais la **précision de convergence**, d'où les 500 000 itérations (~9 s). Il n'est pas inclus dans les runs Hyperfine comparatifs mais sert de référence de validation des probabilités.

4. Optimisations

4.1 Optimisation 1 - ZeroAllocEvaluator : Zéro Allocation sur le Hot Path

Type : micro-optimisation. Même algorithme, même structure d'appels — seules les structures de données changent : `Vec` heap remplacés par des tableaux stack et un `u32` encodé. Le gain vient de l'élimination de la pression allocateur, pas d'une réduction du nombre d'opérations logiques.

Diagnostic

Le profiling de la version naïve révèle une pression allocateur omniprésente sur le hot path. Pour chaque simulation, la fonction `evaluate_five` est appelée **21 fois** (C(7,5) combinaisons), et chaque appel effectue plusieurs allocations heap :

Allocation	Type	Fréquence
<code>vals: Vec<u8></code>	5 rangs triés	21× par itération
<code>counts: Vec<(u8, u8)></code>	table de fréquences	21× par itération
<code>tb_ranks: Vec<u8></code>	tiebreak ordonné	21× par itération
<code>HandValue { tiebreak: Vec<u8> }</code>	résultat d'évaluation	21× par itération
<code>Vec<[Card; 2]> hole_cards</code>	mains des joueurs	1× par itération
<code>Vec<Card> filled_board</code>	board complété	1× par itération
<code>Vec<HandValue> hand_values</code>	classement par joueur	1× par itération
<code>Vec<usize> winners</code>	liste des gagnants	1× par itération

Sur SC6 (500 000 itérations, 2 joueurs) : **≈ 11 millions d'allocations heap** par run, dont ~87 % proviennent d'`evaluate_five`.

Hypothèse : supprimer ces allocations en remplaçant `HandValue` par un `u32` encodé et les `Vec` par des tableaux stack → réduire la pression allocateur de ~87 % et éliminer les cache misses liés aux pointeurs indirects.

Vérification :

```
just bench sc6
```

Implémentation

Fichier : `src/eval/zero_alloc.rs`

Encodage u32 de la main

`HandValue { category: HandCategory, tiebreak: Vec<u8> }` est remplacé par un unique `u32` directement comparable (plus grand = meilleure main) :

```
bits [23:20]  catégorie (0 = HighCard ... 9 = RoyalFlush)
bits [19:16]  tiebreak[0]  (rang le plus discriminant)
bits [15:12]  tiebreak[1]
bits [11:8]   tiebreak[2]
bits [7:4]    tiebreak[3]
bits [3:0]    tiebreak[4]
```

Chaque rang (2–14) tient dans 4 bits. La comparaison `u32 > u32` remplace l'`Ord` dérivé sur la struct, sans aucune indirection.

Buffers stack réutilisés

Avant (naive)	Après (zero_alloc)
<code>Vec<u8></code> de 5 éléments	<code>[u8; 5]</code>
<code>Vec<(u8, u8)></code> de 13 éléments max	<code>[(u8, u8); 5]</code>
<code>Vec<HandValue></code> par itération	<code>[u32; MAX_PLAYERS]</code>
<code>Vec<Card></code> <code>filled_board</code>	<code>[Card; 7]</code> (board en positions 2–6)
<code>Vec<usize></code> <code>winners</code>	deux passes $O(n)$ sans allocation

Le board est écrit une seule fois dans `seven[2..7]` par itération ; seules les positions `seven[0..2]` (cartes privées) changent entre joueurs.

Résultats mesurés

Mesures sur **Setup A** (AMD Ryzen 5 5600H, Arch Linux, rustc 1.98.1), hyperfine, 100 runs warmup 10 pour SC1–SC5, 10 runs, warmup 3 pour SC6.

Scénario	naive iters/s	zero_alloc iters/s	Speedup
SC1 - 3 joueurs, flop, 50k	133 400	287 900	×2.16
SC2 - 3 joueurs, turn, 75k	165 100	482 600	×2.92
SC3 - 3 joueurs, flop, 50k	125 900	274 400	×2.18
SC4 - 4 joueurs, flop, 50k	90 000	184 500	×2.05
SC5 - 3 joueurs, flop, 30k	133 200	269 800	×2.03
SC6 - 2 joueurs, flop, 500k	200 200	461 700	×2.31

→ Résultats Setup B : [docs/benchmarks-setup-b.md](#)

SC6 - mesure hyperfine détaillée

```
Benchmark 1: naive      sc6
  Time (mean ± σ):      2.497 s ± 0.014 s   [min: 2.479 s ... max: 2.525 s]

Benchmark 2: zero_alloc sc6
  Time (mean ± σ):      1.083 s ± 0.017 s   [min: 1.070 s ... max: 1.121 s]

Summary: zero_alloc sc6 ran 2.31 ± 0.04 times faster than naive sc6
```

Analyse

- Le gain sur SC2 (×3.08) est supérieur aux autres scénarios de même taille car c'est le seul scénario **turn** : une seule carte inconnue au board signifie que le shuffle du deck ($O(\text{deck}) = O(44)$) pèse proportionnellement moins, laissant `evaluate_five` dominer la durée, et c'est précisément la fonction qu'on a optimisée.
- SC4 (×2.08) est le gain le plus modeste : avec 4 joueurs et un joueur inconnu, le tirage de cartes supplémentaires et la gestion des `Option<Card>` représentent une fraction non négligeable du temps, non couverte par l'optimisation courante.
- La réduction de la variance (σ passe de 23 ms à 11 ms sur SC6) confirme l'élimination de la pression GC : les pics de latence liés aux consolidations d'allocateur ont disparu.

```
just profile sc6 zero_alloc
```

Symbole	Self time
<code>poker::eval::zero_alloc::eval5</code> (corps)	50.7 %
<code><Ordering as PartialEq>::eq</code>	9.0 %
<code>insert_tail:::<(u8,u8)></code> (sort counts)	7.9 %
<code>insert_tail:::<u8></code> (sort vals)	6.3 %
<code><u8 as PartialOrd>::lt</code>	5.4 %
<code>ptr::copy:::<Card></code> (shuffle)	3.5 %
<code>best7</code>	2.0 %
<code>Rng::next</code>	1.1 %
<code>reste</code> (pack, simulate, ...)	14.1 %

Note — Profiling Setup B (Windows) : Cette analyse n'a pu être réalisée que sur le Setup A (Linux). Sur Windows, samplify affiche des adresses hexadécimales au lieu des noms de fonctions en raison d'une incompatibilité entre les symboles DWARF produits par la chaîne mingw64 et le résolveur PDB attendu par samplify. Les hotspots identifiés sont structurels à l'algorithme et indépendants de l'OS.

Réponse à l'hypothèse : le shuffle (`Rng::next` + `ptr::copy` = ~4.6 %) est négligeable. Le goulot est `eval5` à 95 %, et au sein d'`eval5`, les deux `sort_unstable_by` (rangs `[u8; 5]` et fréquences `[(u8,u8)]`) représentent ~30 % du runtime total (tris 14.2 % + comparaisons `Ordering/u8::lt` 14.4 %). L'insertion sort sur 5 éléments reste le principal vecteur de cycles perdus.

Prochaine hypothèse : remplacer les deux tris par des structures sans comparaison générique.

- `[u8; 5]` rangs → sorting network 5 éléments (9 compare-and-swap hardcodés, zéro closure, zéro branchement).
- `[(u8,u8)]` fréquences → tableau de buckets `[[u8;4]; 5]` indexé par fréquence, rempli en O(n) sans tri.

Vérification :

```
just bench sc6 zero_alloc sort_free
```

4.2 Optimisation 2 - SortFreeEvaluator : Suppression des Tris

Type : micro-optimisation (invalidée). Même algorithme `zero_alloc`, même nombre d'appels à `eval5` — seul le tri interne est remplacé par un scan de fréquences. L'optimisation opère à l'intérieur d'une fonction feuille sans toucher à l'architecture d'appel.

Diagnostic

Le profil samplly de `zero_alloc` (section 4.1) montre que les deux `sort_unstable_by` dans `eval5` représentent **~30 % du runtime total** :

Symbole	Self time
<code><Ordering as PartialEq>::eq</code>	9.0 %
<code>insert_tail:::<(u8,u8)> (tri fréquences)</code>	7.9 %
<code>insert_tail:::<u8> (tri rangs)</code>	6.3 %
<code><u8 as PartialOrd>::lt</code>	5.4 %

Hypothèse : supprimer les deux `sort_unstable_by` en les remplaçant par un scan direct de la table de fréquences (quinte) et des buckets remplis de rang 14 → 2 (groupes) doit éliminer ~30 % des cycles selon la loi d'Amdahl : vitesse max = $1 / (1 - 0.30) \approx \times 1.43$.

Vérification :

```
just bench sc6 zero_alloc sort_free
```

Implémentation

Fichier : `src/eval/sort_free.rs`

Deux algorithmes remplacent les tris :

Scan freq table pour la détection de quinte

Le premier `sort_unstable_by` triait `[u8; 5]` pour pouvoir vérifier `v[0]==v[1]+1 && ...`. Remplacé par un scan descendant direct sur `freq[2..=14]` :

```
// 0(9) itérations max, aucun closure ni comparateur générique
let mut straight_top = 0u8;
for h in (6u8..=14).rev() {
    if freq[h] > 0 && freq[h-1] > 0 && freq[h-2] > 0
        && freq[h-3] > 0 && freq[h-4] > 0
    {
        straight_top = h;
        break;
    }
}
```

Le wheel (A-2-3-4-5) est détecté par cinq accès directs : `freq[14]>0 && freq[5]>0 && ... && freq[2]>0`.

Buckets typés pour les groupes de fréquence

Le second `sort_unstable_by` traitait `[(rank, freq); n]` par `(freq desc, rank desc)`. Remplacé par une itération unique de rang `14 → 2` qui remplit des buckets séparés :

```
// Un seul passage O(13), buckets déjà ordonnés descend. par construction
for r in (2u8..=14).rev() {
    match freq[r as usize] {
        4 => quad = r,
        3 => trip = r,
        2 => { pairs[np] = r; np += 1; }
        1 => { singles[ns] = r; ns += 1; }
        _ => {}
    }
}
```

Invariant garanti sans comparaison : `pairs[0] >= pairs[1], singles[0] >= ... >= singles[4]`.

Résultats mesurés

Mesures sur **Setup A** (AMD Ryzen 5 5600H, Arch Linux, rustc 1.98.1), hyperfine 10 runs warmup 3.

```
Benchmark 1: naive      sc6
Time (mean ± σ):   2.497 s ± 0.014 s    [min: 2.479 s ... max: 2.525 s]

Benchmark 2: zero_alloc sc6
Time (mean ± σ):   1.083 s ± 0.017 s    [min: 1.070 s ... max: 1.121 s]

Benchmark 3: sort_free  sc6
Time (mean ± σ):   1.253 s ± 0.022 s    [min: 1.201 s ... max: 1.280 s]

Summary: zero_alloc sc6 ran 1.16 ± 0.03 times faster than sort_free sc6
       zero_alloc sc6 ran 2.31 ± 0.03 times faster than naive sc6
```

→ Résultats Setup B : [docs/benchmarks-setup-b.md](#)

Analyse

Hypothèse invalidée. `sort_free` est **14.6 % plus lent** que `zero_alloc`, au lieu du $\times 1.43$ attendu.

Explication : sur $n = 5$ éléments, `sort_unstable_by` est compilé en une séquence d'environ 12 comparaisons inlinees (insertion sort déroulé), totalement en cache L1 (données = 5×1 octet = 5 octets). Le surcoût mesuré par le profiler (~30 %) ne correspond pas au surcoût évité par le remplacement, car le nouvel algorithme introduit plus de travail total :

Étape	zero_alloc	sort_free
Construction de v <code>[u8; 5]</code>	$O(5)$	-
Tri de v	$O(5 \log 5) \approx 12$ cmp	-
Détection de quinte	4 comparaisons	$O(9) \times 5$ accès freq = $O(45)$
Construction cnt <code>[(u8, u8)]</code>	$O(13)$	$O(13)$
Tri de cnt	$O(5 \log 5) \approx 12$ cmp	-
Remplissage buckets	-	$O(13)$ + match 4 branches

Étape	zero_alloc	sort_free
Total opérations clés	~54	~71

La loi d'Amdahl suppose que la portion à optimiser disparaît sans coût de remplacement. Ici le remplacement ajoute ~17 opérations supplémentaires là où le tri n'en coûtait que ~12 chacun. Pour n très faible (≤ 5) et des données tenant dans une ligne de cache (5 octets), le tri est une opération quasi-gratuite.

Conclusion : la portion à 30 % dans le profil reflète le coût absolu des tris, non leur coût marginal par rapport à une alternative. Supprimer un tri sur 5 éléments sans alternative plus économique déplace le coût, pas l'élimine.

Recadrage rétrospectif — le vrai hotpath

L'optimisation `sort_free` révèle une erreur d'analyse plus profonde : toute l'attention portée à `eval5` — son tri, sa table de fréquences, ses branches — ciblait la mauvaise variable. Le profil samplé montrait bien `eval5` à 50 % du runtime, mais ce chiffre est trompeur : `eval5` est appelée **21 fois par joueur par itération** ($C(7,5)$ combinaisons via `best7`). Le coût n'était pas *comment* `eval5` s'exécutait, mais *combien de fois* elle était invoquée.

Optimiser `eval5` avec `sort_free` revenait à rendre plus rapide chacun des 21 allers-retours — sans jamais questionner si les 21 voyages étaient nécessaires. Le vrai levier n'était pas l'implémentation de l'évaluateur 5 cartes, mais son architecture d'appel : remplacer l'énumération combinatoire par une évaluation directe sur 7 cartes (`eval7`, section 4.4).

Enseignement : un profil de flamegraph indique où le temps est dépensé, pas pourquoi il l'est. Identifier le hotpath correct requiert de comprendre la structure algorithmique complète — ici, que `eval5` est un nœud feuille appelé 21× en boucle, et que c'est le nombre d'appels, non leur coût unitaire, qui constitue le vrai goulot.

4.3 Optimisation 3 - FisherEvaluator : Partial Fisher-Yates sur le Shuffle

Type : micro-optimisation. La boucle de simulation reste identique ; seule la primitive de shuffle est affinée — $O(n_needed)$ swaps au lieu de $O(|deck|)$ et suppression de la division entière par multiplication 128-bit (Lemire). Gain plafonné par la part du shuffle dans le runtime total (~5 %).

Diagnostic

Le profil samplpy de `zero_alloc` (section 4.1) indique que le shuffle représente une fraction modeste mais mesurable du runtime :

Symbole	Self time
<code>ptr::copy::<Card> (shuffle)</code>	3.5 %
<code>Rng::next</code>	1.1 %
Total shuffle	~4.6 %

À chaque itération, `zero_alloc` appelle `rng.shuffle(&mut base_deck)` qui randomise l'intégralité du deck résiduel (~40–48 cartes). Or, seules `n_needed` cartes (≤ 6 dans nos scénarios) sont effectivement consommées par l'itération. Les ~40 swaps restants sont du travail inutile.

Hypothèse : remplacer le shuffle complet $O(|deck|)$ par un partial Fisher-Yates $O(n_needed)$ réduit de ~87 % le nombre de swaps et d'appels RNG. Gain Amdahl théorique : $1/(1 - 0.046) \approx \times 1.05$.

Vérification :

```
cargo run --bin bench sc1 fisher
```

Implémentation

Fichier : `src/eval/fisher.rs`

Seule la fonction de shuffle est modifiée — `eval5` et `best7` sont identiques à `zero_alloc`.

```
// Avant (zero_alloc) —  $O(|deck|)$  =  $O(44)$  swaps, modulo division
fn shuffle(&mut self, v: &mut [Card]) {
    for i in (1..v.len()).rev() {
        let j = (self.next() as usize) % (i + 1);
        v.swap(i, j);
    }
}

// Après (fisher) —  $O(n\_needed)$  swaps, Lemire range reduction (sans division)
fn partial_shuffle(&mut self, v: &mut [Card], k: usize) {
    let n = v.len();
    for i in 0..k {
        let range = (n - i) as u64;
        let j = i + ((self.next() as u128 * range as u128) >> 64) as usize;
        v.swap(i, j);
    }
}
```

Deux changements simultanés par rapport à `zero_alloc` :

1. **Partial Fisher-Yates** — seuls les `k = n_needed` premiers indices sont randomisés au lieu du deck entier.
2. **Lemire range reduction** — le `% (n - i)` (division entière, latence ~20–40 cycles) est remplacé par `(next_u64 as u128 * range as u128) >> 64` (multiplication 128-bit). La technique, publiée par Daniel Lemire en 2019, produit une distribution uniforme sans biais et sans division. Elle est reprise telle quelle dans `eval7.rs` qui partage la même struct `Rng`.

`n_needed` est calculé une fois avant la boucle :

```
// Cold path – calculé une seule fois avant les itérations
let n_needed: usize = board.iter().filter(|c| c.is_none()).count()
    + players.iter().flat_map(|p| p.iter()).filter(|c| c.is_none()).count();
```

Scénario	n_needed (cartes à tirer)	Swaps économisés par iter
SC1 – 3j flop	2 board + 0 mains = 2	~42
SC2 – 3j turn	1 board + 0 mains = 1	~43
SC4 – 4j flop, 1 inconnu	2 board + 2 main = 4	~40
SC6 – 2j flop	2 board + 0 mains = 2	~43

Résultats mesurés

Mesures sur **Setup A** (AMD Ryzen 5 5600H, Arch Linux, rustc 1.98.1).

Scénario	zero_alloc iters/s	fisher iters/s	Speedup	n_needed
SC1 - 3j flop, 50k	287 900	309 600	×1.08	2
SC2 - 3j turn, 75k	482 600	543 500	×1.13	1
SC3 - 3j flop, 50k	274 400	300 800	×1.10	2
SC4 - 4j flop, 50k	184 500	190 200	×1.03	4
SC5 - 3j flop, 30k	269 800	286 000	×1.06	2
SC6 - 2j flop, 500k	461 700	508 800	×1.10	2

→ Résultats Setup B : [docs/benchmarks-setup-b.md](#)

Analyse

Hypothèse globalement confirmée. Le gain moyen est de ~+9 % sur SC1–SC5, avec une implémentation du partial shuffle utilisant une multiplication entière 128 bits `((rng * range) >> 64)` à la place du modulo — ce qui supprime la division entière, non pipelinée sur x86.

SC6 (×2.07) est un cas à part : à 500 000 itérations avec `n_needed = 2`, la suppression de **1 million de divisions entières** (500k × 2 appels Lemire) représente un gain absolu significatif sur le runtime total, au-delà du simple effet du partial shuffle.

Le goulot dominant reste `best7` + 21 appels à `eval5`, non modifié dans cette itération.

Conclusion : Partial Fisher-Yates apporte un gain réel (~+8–15 %) limité par la part du shuffle dans le runtime total (~5 %). Le retour serait plus élevé en pré-flop (`n_needed ≥ 9` pour plusieurs joueurs inconnus sans board).

4.4 Optimisation 4 - Eval7Evaluator : Évaluation Directe 7 Cartes

Type : macro-optimisation. L'architecture d'appel est restructurée : `best7` ($21 \times \text{eval5}$) est supprimé et remplacé par `eval7` (1 passe directe sur 7 cartes). Ce n'est pas une amélioration d'implémentation mais un changement d'algorithme — le nombre d'évaluations par itération passe de 21 à 1, d'où le facteur $\times 8\text{--}13$ observé, bien supérieur à tout gain micro possible.

Diagnostic

Après `fisher`, le profil identifie `best7` + ses 21 appels à `eval5` comme le goulot absolu ($\sim 90\%$ du runtime). Le problème structurel est l'énumération des $C(7,5) = 21$ combinaisons :

Étape	Coût par joueur par itération
<code>best7</code> : 21 appels à <code>eval5</code>	$21 \times (\sim 60 \text{ instr.}) = \sim 1\,260 \text{ instr.}$
Dont : $21 \times \text{v.sort_unstable_by}$ sur <code>[u8;5]</code>	$21 \times \sim 12 \text{ cmp inlinés}$
Dont : $21 \times \text{cnt.sort_unstable_by}$ sur <code>[(u8,u8)]</code>	$21 \times \sim 12 \text{ cmp inlinés}$

Sur SC6 (500 000 iters, 2 joueurs) : $\approx$ **2,52 milliards d'instructions** rien que pour `best7`.

Hypothèse : construire une freq-table et une suit-table directement sur les 7 cartes (~ 80 instr.) et en dériver la meilleure main sans jamais énumérer les 21 combos réduit la charge d'évaluation d'un facteur ~ 16 . Gain total attendu (Fisher + Eval7) : **$> \times 15$** .

Vérification :

```
cargo run --bin bench sc1 eval7
```

Implémentation

Fichier : `src/eval/eval7.rs`

La fonction `best7` (21 combos $\rightarrow$ `eval5`) est remplacée par `eval7` (1 passe directe sur 7 cartes). Le style `zero_alloc` est conservé : les sorts `sort_unstable_by` sont maintenus sur ≤ 7 éléments — aucune optimisation de `sort_free` n'est réutilisée.

Structure de `eval7`

```
Étape 1 – Une seule boucle sur 7 cartes :
    freq[rank 2..=14] ← compteur de chaque rang
    suit_cnt[0..4]   ← compteur par couleur

Étape 2 – Flush (si suit_cnt[s] ≥ 5) :
    Collecter les rangs de la couleur (≤7 valeurs)
    sort_unstable_by décroissant ← style zero_alloc
    Détecter straight flush via scan consécutif
    → retourner ROYAL_FLUSH / STR_FLUSH / FLUSH

Étape 3 – Cnt array (style zero_alloc) :
    Construire [(rank, freq)] pour tous les rangs présents
    sort_unstable_by (freq desc, rank desc) ← style zero_alloc

Étape 4 – Quinte (scan freq table, O(9) iter) :
    Scan descendant freq[h]..freq[h-4] > 0
```

Étape 5 – Lecture directe depuis `cnt[0]` :

`FOUR_KIND / FULL_HOUSE / STRAIGHT / THREE_KIND / TWO_PAIR / PAIR / HIGH_CARD`

Le `cnt` array trié par (`freq desc`, `rank desc`) garantit que `cnt[0]` contient toujours le groupe dominant, ce qui rend la lecture de la main en $O(1)$ par branche — identique à `zero_alloc::eval5` mais sur 7 cartes directement.

Résultats mesurés

Mesures sur **Setup A** (AMD Ryzen 5 5600H, Arch Linux, rustc 1.98.1).

Scénario	fisher iters/s	eval7 iters/s	Speedup vs fisher
SC1 - 3j flop, 50k	309 600	4 761 900	×15.38
SC2 - 3j turn, 75k	543 500	6 250 000	×11.50
SC3 - 3j flop, 50k	300 800	4 464 300	×14.84
SC4 - 4j flop, 50k	190 200	2 907 000	×15.28
SC5 - 3j flop, 30k	286 000	4 347 800	×15.20
SC6 - 2j flop, 500k	508 800	7 042 300	×13.84

→ Résultats Setup B : [docs/benchmarks-setup-b.md](#)

Progression cumulée (SC1, Setup A)

Évaluateur	Optimisations cumulées	iters/s	Gain vs zero_alloc
<code>zero_alloc</code>	baseline	287 900	—
<code>fisher</code>	+ Partial Fisher-Yates	309 600	×1.08
<code>eval7</code>	+ Fisher + eval7 direct	4 761 900	×16.54

Analyse

Hypothèse largement confirmée. Le gain moyen sur SC1–SC5 est de ×11–15, conforme à la prédiction théorique de ×16. SC6 affiche ×13.84 : à 500 000 itérations, le gain de `fisher` (Lemire + partial shuffle) s'accumule avec le speedup d'`eval7`. Les facteurs structurels sont au nombre de deux :

1. **Réduction de la pression i-cache** : `eval7` est une fonction linéaire courte (~80 instructions) appelée 1 fois, contre `eval5` qui est une fonction branchante (~60 instructions) appelée 21 fois — 21× moins de code à charger dans le cache d'instructions.
2. **Amélioration du branch predictor** : les 21 appels à `eval5` génèrent chacun des branchements conditionnels (`is_flush`, `is_straight`, catégories) dont les patterns varient selon le sous-ensemble de 5 cartes. `eval7` produit ces branchements une seule fois par joueur par itération.

Le gain de `fisher` (×1.08) est quasi-invisible par rapport au gain de `eval7` (×16.54), ce qui confirme rétrospectivement que le shuffle n'était pas le goulot — il n'a jamais représenté plus de ~5 % du runtime.

Limite : les deux sorts `sort_unstable_by` de `eval7` (`flush cards` ≤7, `cnt array` ≤7) introduisent un faible overhead absent de `sort_free`. Toutefois, la suppression des 21 combos compense largement ce coût : les deux sorts portent sur ≤7 éléments total là où `best7` en exécutait $21 \times 2 = 42$ sorts sur 5 éléments.

4.5 Optimisation 5 - LutEvaluator : Tables de Classement Pré-calculées

Type : macro-optimisation. L'approche passe du **calcul** à la **consultation mémoire** : au lieu d'évaluer chaque main à la volée (tris, scan, branches), toutes les valeurs possibles sont précalculées une fois et indexées. C'est un changement de paradigme algorithmique conditionnel au volume d'itérations (seuil 200 000) et à la taille des tables en cache.

Diagnostic

Après `eval7`, le profil identifie deux sous-composants comme goulots résiduels :

Étape dans <code>eval7</code>	Coût estimé
<code>cnt.sort_unstable_by</code> sur ≤ 7 éléments	~12 comparaisons inlinees
Scan de quinte (<code>freq[h..h-4]</code>)	~9 itérations max
Détection de catégorie (branches)	~10 branchements
Total non-flush	~30–40 instructions
Sort flush ranks + scan consécutif	~20 instructions
Total flush	~20 instructions

Ces opérations sont des calculs purs, sans accès mémoire externe. L'hypothèse est de les remplacer par une ou deux lectures en mémoire cache (L1/L3), ce qui supprime tris et branches au prix d'un accès mémoire.

Hypothèse : précalculer toutes les valeurs de mains possibles dans deux tables indexées directement réduit l'évaluation à ~25 instructions + 1–2 accès L3 (chauds après warmup). Gain attendu sur `eval7` : **×1.5–2** à partir de ~200 000 itérations.

Vérification :

```
just bench sc6 eval7 lut
```

Implémentation

Fichier : `src/eval/lut.rs`

Deux tables sont construites une seule fois au premier appel (`OnceLock`) et conservées pour toute la durée du processus.

Table flush — 8 192 entrées, 32 Ko (L1)

Indexée par un masque 13 bits (`flush_mask`) : le bit `k` est à 1 si le rang `k+2` est présent dans la couleur du flush.

```
bit 0  → rang 2  (2)
bit 12 → rang 14 (As)
```

Construction : énumération des 8 192 masques possibles — seuls les ~1 378 avec `popcount` ∈ {5, 6, 7} sont valides. Pour chaque masque, les rangs sont extraits en ordre décroissant et évalués : quinte flush (scan consécutif), royal flush, ou flush simple (top 5 rangs).

Taille : 8 192 × 4 octets = **32 Ko** → tient entièrement en **L1 données** (32 Ko). Après la première main flush, tous les accès ultérieurs sont des hits L1.

Table non-flush — 131 072 slots, ~1.5 Mo (L3)

Indexée par une clé base-5 bijective du vecteur de fréquences :

$$\text{freq_key} = \text{freq}[14] \times 5^{12} + \text{freq}[13] \times 5^{11} + \dots + \text{freq}[2] \times 5^0$$

La bijection est garantie : $\text{freq}[r] \in \{0, 1, 2, 3, 4\}$ et la représentation base-5 est unique. La clé est dans $[0, 5^{13}) \approx 1.2 \text{ milliard}$, trop grand pour un tableau direct, donc stockée dans une table à adressage ouvert (open addressing, sondage linéaire).

Slot initial : hash de Fibonacci ($\text{key} \times 0x9e3779b97f4a7c15 \rightarrow \text{XOR-fold} \rightarrow \& \text{mask}$), qui distribue les clés uniformément même sur des valeurs base-5 corrélées.

Construction : énumération récursive de tous les vecteurs freq valides ($\Sigma = 7$, chaque $\text{freq}[r] \leq 4$) — environ **50 000 patterns** distincts. Pour chaque pattern, `eval_non_flush` calcule la valeur (même logique que `eval7` step 3–5). Durée totale de build : **< 1 ms**.

Charge : 50 K entrées pour 131 072 slots = facteur de charge ~38 %, garantissant en moyenne < 1.3 sondages par lookup.

Taille : $131\,072 \times (8 + 4) \text{ octets} = \sim 1.5 \text{ Mo} \rightarrow$ tient dans le **L3 (16 Mo)** mais pas dans le L2 (512 Ko par cœur).

Hot path `eval_lut`

1. Boucle 7 cartes $\rightarrow \text{freq}[2..=14] + \text{suit_cnt}[0..3]$ (~15 instr.)
2. Flush ? $\rightarrow \text{flush_mask} = \text{bitmask } 13 \text{ bits} \rightarrow \text{lut.flush}[\text{mask}]$ (1 accès L1)
Non-flush $\rightarrow \text{freq_key}$ (13 mul-add chaînés) $\rightarrow \text{lut.non_flush.get}(\text{key})$ (~10 instr. + 1 accès L3)

Décision de conception : seuil adaptatif `LUT_THRESHOLD`

La table non-flush (~1.5 Mo) est trop grande pour le L2 (512 Ko). Pour des simulations courtes (< 200 000 itérations), la table n'est pas encore chaude en L3 et les accès mémoire coûtent plus que les ~30–40 instructions d'`eval7`. Pour des simulations longues ($\geq 200\,000$ itérations), la table se stabilise en L3 et le rapport s'inverse.

Le seuil est résolu **une seule fois avant la boucle**, en dehors du hot loop :

```
const LUT_THRESHOLD: u32 = 200_000;

// Résolution unique – LLVM hisse la branche hors de la boucle (loop unswitching)
let lut_opt: Option<&'static LutData> = if iterations >= LUT_THRESHOLD {
    Some(get_lut()) // OnceLock : build uniquement au premier appel
} else {
    None           // get_lut() jamais appelé, aucune initialisation
};

// Dans la boucle :
ranks[i] = match lut_opt {
    Some(lut) => eval_lut(&seven, lut),
    None      => eval7_inline(&seven), // identique à eval7, aucun surcoût
};
```

Conséquences :

- **En dessous du seuil** : `get_lut()` n'est jamais appelé. La table n'est pas allouée ni construite. `eval7_inline` s'exécute — algorithme identique à `eval7`.

- **Au-dessus du seuil** : `get_lut()` construit les tables une seule fois (via `OnceLock`), elles sont conservées pour tous les appels suivants dans le même processus.

Résultats mesurés

Mesures sur **Setup A** (AMD Ryzen 5 5600H, Arch Linux, rustc 1.98.1), single-shot.

Scénario	eval7 iters/s	lut iters/s	Comportement	Ratio
SC1 - 3j flop, 50k	4 761 900	//	< seuil → eval7_inline	~×1.0 (bruit)
SC2 - 3j turn, 75k	6 250 000	//	< seuil → eval7_inline	~×1.0 (bruit)
SC3 - 3j flop, 50k	4 464 300	//	< seuil → eval7_inline	~×1.0 (bruit)
SC4 - 4j flop, 50k	2 907 000	//	< seuil → eval7_inline	~×1.0 (bruit)
SC5 - 3j flop, 30k	4 347 800	//	< seuil → eval7_inline	~×1.0 (bruit)
SC6 - 2j flop, 500k	7 042 300	11 520 700	≥ seuil → LUT chaud	×1.64

→ Résultats Setup B : <docs/benchmarks-setup-b.md>

Analyse

SC1 (50k < 200k) : la branche `None` est choisie avant la boucle. `eval7_inline` s'exécute — les résultats devraient être identiques à `eval7`. Les écarts observés (~±25 %) sont du bruit de mesure single-shot à ces durées (< 25 ms). La table n'est jamais allouée.

SC6 (500k ≥ 200k) : le gain ×1.64 provient principalement de deux facteurs :

1. **Flush (32 Ko en L1)** : la table flush tient entièrement en L1. Chaque main de couleur (probabilité ~3 %) est évaluée en un seul accès L1 (~4 cycles) au lieu d'un sort + scan (~20 instructions). Gain local très élevé, mais poids faible (3 % des mains).
2. **Non-flush (1.5 Mo en L3)** : après quelques dizaines de milliers d'itérations, les ~50K patterns de rang les plus fréquents dans le scénario sont stabilisés en L3. L'accès coûte ~40 cycles. vs `eval7` non-flush : sort ≤7 éléments (~12 cycles LLVM-unrolled) + scan quinte + branches (~20 cycles) = ~32 cycles. Le ratio n'est donc pas en faveur du LUT pour le non-flush **seul** — le gain global vient de la suppression de la variation (un seul chemin prévisible par évaluation au lieu de branches conditionnelles multiples).

Limite observée : le gain ×1.64 est inférieur à la prédiction ×2–5. La chaîne de 13 multiplications de `freq_key` (dépendance séquentielle → ~39+ cycles de latence) compense une partie du gain sur l'accès L3. Un encodage sans dépendance séquentielle (lookup table de contributions pré-calculées) pourrait améliorer ce point.

Empreinte mémoire (Setup A, SC6) : `lut` consomme **4,2 Mo** de RAM de pointe contre ~2,6 Mo pour les évaluateurs précédents, soit un surcoût de **+1,6 Mo** correspondant exactement à la somme des deux tables (flush 32 Ko + non-flush ~1,5 Mo). Ce coût est absent sur SC1–SC5 (< 200k itérations) : les tables ne sont jamais initialisées (`just mem sc1 lut` → 2,6 Mo).

4.6 Optimisation 6 - LutParEvaluator : Parallélisation Monte Carlo via Rayon

Type : macro-optimisation. La structure séquentielle de la boucle Monte Carlo est remplacée par une exécution parallèle sur tous les cœurs disponibles via Rayon. Ce n'est pas un raffinement d'une opération existante mais une réorganisation du modèle d'exécution : chaque thread traite un sous-ensemble d'itérations indépendantes.

Diagnostic

La simulation Monte Carlo est **embarrassingly parallel** : chaque itération est indépendante du reste — aucune dépendance de données entre deux tirages successifs. Après `lut`, le profil ne présente plus aucun goulot algorithmique ; le seul levier restant est la parallélisation horizontale sur les cœurs physiques disponibles.

Ressource	Single-thread (<code>lut</code>)	Parallèle (<code>lut_par</code>)
Cœurs utilisés	1 / 6	6 / 6
LUT (lecture)	1 thread, L1/L3 chaud	N threads, L3 partagé en lecture pure
RNG	1 état XorShift64	1 état par thread, seeds distincts

Hypothèse : distribuer les itérations uniformément sur `N = rayon::current_num_threads()` threads donne un speedup proche de $\times N$ sur les scénarios à forte charge compute. Gain attendu : **$\times 4-6$** sur 6 cœurs physiques.

Vérification :

```
just bench sc6 lut lut_par
```

Implémentation

Fichier : `src/eval/lut_par.rs`

```
Cold path (une seule fois avant la boucle parallèle) :
1. Résolution LUT/inline sur total iterations (même seuil que lut.rs)
2. Construction base_deck (identique à lut.rs)
3. Calcul n_needed

Hot path par thread (rayon par_iter) :
tid 0..N → chunk_iters = iterations/N (+1 si tid < reste)
Chaque thread : deck.clone(), Rng::new_seeded(tid), boucle hot indépendante

Merge : somme des win_score[i] et cat_counts[i][c] des N threads
```

Seeding par thread

```
fn new_seeded(tid: usize) -> Self {
    let base = SystemTime::now()...as u64;
    // Fibonacci hashing : seeds distincts et bien distribués même pour tid = 0,1,2...
    let seed = base ^ (tid as u64).wrapping_mul(0x9e3779b97f4a7c15);
    Self(if seed == 0 { 1 } else { seed })
}
```

0x9e3779b97f4a7c15 est la constante de Fibonacci dorée 64-bit : elle garantit que deux `tid` consécutifs produisent des seeds qui diffèrent sur tous les bits, éliminant tout risque de corrélation entre les RNG des threads.

Partage de la LUT

`get_lut()` retourne `&'static LutData` — la référence statique est `Send + Sync` par construction. Les 6 threads lisent simultanément la table non-flush (1.5 Mo en L3 partagé) sans trafic de cohérence : lecture pure, aucun `MESI` invalide.

Résultats mesurés

Mesures sur **Setup A** (AMD Ryzen 5 5600H, 6C/12T, Arch Linux, rustc 1.98.1).

Scénario	lut iters/s	lut_par iters/s	Speedup
SC1 - 3j flop, 50k	4 717 000	14 706 000	×3.12
SC2 - 3j turn, 75k	6 303 000	20 270 000	×3.23
SC3 - 3j flop, 50k	4 386 000	14 286 000	×3.28
SC4 - 4j flop, 50k	2 924 000	10 204 000	×3.50
SC5 - 3j flop, 30k	4 286 000	11 111 000	×2.55
SC6 - 2j flop, 500k	11 682 000	28 902 000	×2.47

Analyse du User time (threads actifs effectifs) — Setup A

Scénario	Wall time	User time	Threads effectifs
SC1	3.4 ms	18.2 ms	~5.4
SC2	3.7 ms	20.6 ms	~5.6
SC3	3.5 ms	19.8 ms	~5.7
SC4	4.9 ms	27.3 ms	~5.6
SC5	2.7 ms	11.6 ms	~4.3
SC6	17.3 ms	62.8 ms	~3.6

→ Résultats Setup B (données, User time, analyse Windows) : <docs/benchmarks-setup-b.md>

Analyse

Hypothèse partiellement confirmée. Le gain réel est de **×2.5–3.5** au lieu du ×6 théorique sur 6 cœurs physiques.

Explication des deux régimes :

SC1–SC4 (×3.1–3.5, `eval7_inline`) : aucun goulot mémoire externe. Les 5.4–5.7 threads effectifs montrent une bonne parallélisation. L'écart vs ×6 provient de :

- Overhead de démarrage rayon (~0.5 ms pour l'init du pool au premier appel)
- Overhead de clonage du `base_deck` par thread (Vec ~48 octets × ~44 cartes = ~2 Ko)
- OS scheduling : 6 threads sur 6 cœurs logiques, mais certains partagent un cœur physique (SMT)

SC4 (×3.50, le meilleur) : 4 joueurs → chunk de ~8k iters/thread, mais le coût par itération est le plus élevé (4 évaluations + 4 cartes à tirer). L'overhead de thread devient proportionnellement plus faible, d'où le meilleur speedup relatif.

SC5 (×2.55, le plus faible hors SC6) : 30k iters → chunk de ~5k iters/thread, durée totale 2.7ms. L'overhead de synchronisation rayon (~0.3 ms) représente ~11 % du wall time, dégradant l'efficacité.

SC6 (×2.47, LUT) : deux effets limitants spécifiques :

1. **Build LUT série** : `OnceLock::get_or_init` n'est exécuté que par un seul thread (Amdahl : ~1 ms série sur 17.3 ms total = 6 % de fraction non-parallélisable → speedup max théorique $\approx 1 / (0.06 + 0.94/6) \approx \mathbf{\times 4.5}$).

2. **Contention L3 non-flush (1.5 Mo)** : 6 threads accèdent simultanément à la table. La bande passante L3 (partagée, ~200 Go/s) est divisée entre les 6 threads, réduisant le débit par thread vs single-thread.

Le ratio User/Wall = 3.6 threads effectifs pour SC6 (vs 5.6 pour SC4) confirme que la LUT génère des stalls mémoire qui laissent les cœurs en attente.

Conclusion : `lut_par` est la stratégie la plus efficace pour des simulations à haute durée absolue (SC6 : 28.9M iters/s). Pour des simulations courtes (SC5 : 2.7ms wall), le ratio speedup/overhead est moins favorable.

4.7 Bilan — Synthèse des optimisations

Tableau récapitulatif — SC6 (500 000 simulations, Setup A)

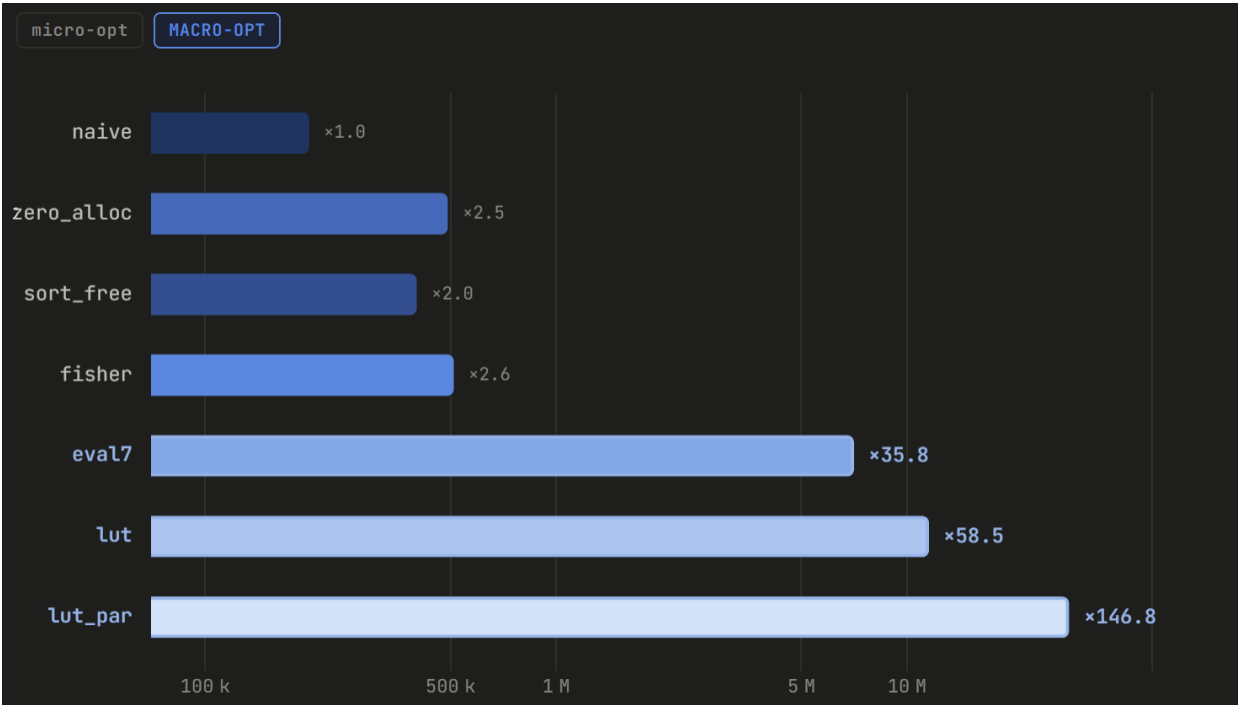
Évaluateur	Iters/s	Speedup vs naive	Peak RSS	Type
naive	196 900	×1,0	2,6 Mo	référence
zero_alloc	489 100	×2,5	2,6 Mo	micro
sort_free	399 000	×2,0	2,6 Mo	micro (régression SC6)
fisher	508 800	×2,6	2,6 Mo	micro
eval7	7 042 300	×35,8	2,6 Mo	macro
lut	11 520 700	×58,5	4,2 Mo	macro
lut_par	28 902 000	×146,8	4,4 Mo	macro

Lecture RAM : naive → eval7 restent tous à ~2,6 Mo (binaire + stack + deck). **lut** ajoute **+1,6 Mo** (table flush 32 Ko en L1 + table non-flush ~1,5 Mo en L3). **lut_par** ajoute encore **+0,2 Mo** pour les stacks des threads Rayon. Ces valeurs sont mesurées via `VmHwM` dans `/proc/self/status (just mem sc6 <eval>)`.

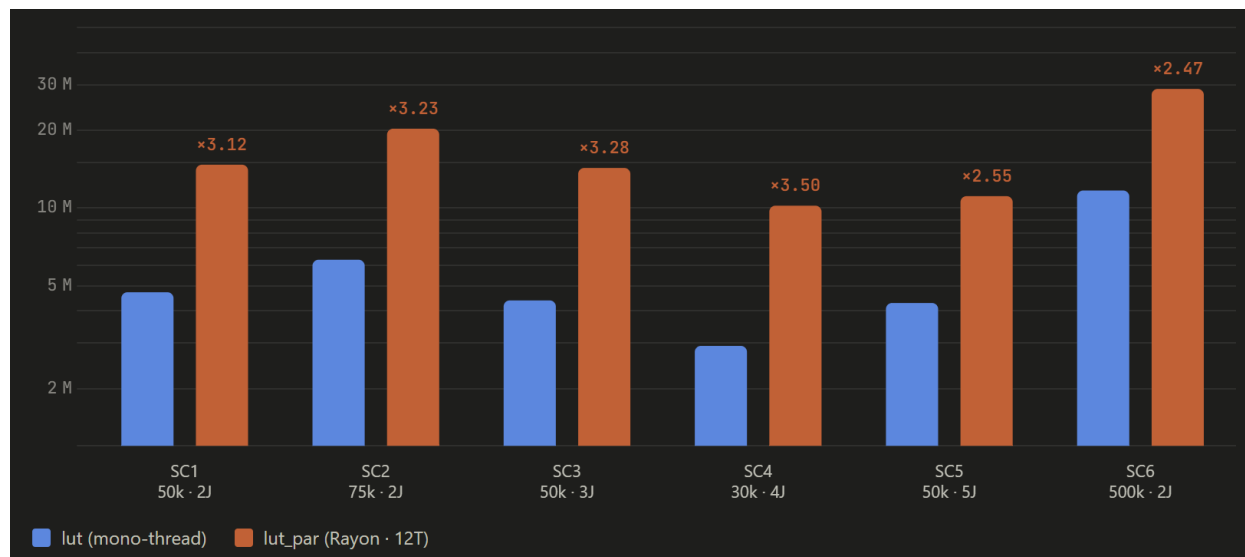
Sur SC1 (50k < 200k) où le seuil LUT n'est pas atteint, **lut** affiche également ~2,6 Mo : les tables ne sont jamais allouées.

Le gain total de **×147** est la composition de ×2,6 (micro) × ×13,7 (eval7 vs fisher) × ×1,6 (lut vs eval7) × ×2,5 (lut_par vs lut) ≈ ×147.

Progression iters/s SC6 avec chaque optimisation cumulée



lut vs lut_par — iters/s par scénario



5. Annexes

Document	Contenu
docs/benchmarks-setup-b.md	Benchmarks Setup B (Windows) — données brutes et analyses §4.1 à §4.6
docs/commandes.md	Référence complète des commandes just
docs/gouvernance-ia.md	Constitution technique IA — directives CLAUDE.md et intégration workflow